

1 Aufgabe 2

Die Messungen wurden auf einer NVIDIA GeForce RTX 4060 durchgeführt. Laut Geräteabfrage liegt die theoretische Speicherbandbreite bei 272 GB/s. Die GPU besitzt 24 SMs und maximal 1536 Threads pro SM, also 48 Warps pro SM.

1.1 Speicherintensiver Kernel

Ein speicherintensiver Kernel wurde in `memory.cu` implementiert. Jeder Thread liest einen Wert aus `a` multipliziert diesen mit einer Konstanten `c` und schreibt diesen Wert nach `b`.

```
b[i] = a[i] * c;
```

Während der Rechenaufwand pro Element mit höchstens 0,25 FLOP/Byte sehr gering ist, entstehen Speicherzugriffe im Umfang von 8 Byte durch einen 4 Byte *read* und einen 4 Byte *write*. Damit wird die Laufzeit des Kernels primär durch das Speicherzugriffsmuster und nicht die ALUs bestimmt.

Es wurden insgesamt drei Varianten des Kernels implementiert:

- `kernel_copy_coalesced`: benachbarte Threads lesen benachbarte Adressen.
- `kernel_copy_strided`: Thread `i` liest `a[(i * stride) % n]`.
- `kernel_copy_gather`: Thread `i` liest eine zufällige Adresse `a[idx[i]]`.

1.1.1 STREAM-Triad als Referenzfall

Zusätzlich wurde ein STREAM-Kernel in `stream.cu` als Referenz implementiert. Dieser bearbeitet ein klassisches Bandbreitenproblem:

```
a[i] = b[i] + alpha * c[i];
```

Pro Element werden zwei Werte `b[i]` und `c[i]` gelesen und ein Wert `a[i]` geschrieben. Das erzeugt bei FP32 Speicherzugriffe im Umfang von 12 Bytes für nur eine *fused multiply add (fma)*. Diese hat einen arithmetischen Aufwand von etwa 0,17 FLOP/Byte.

Der Unterschied zu `memory.cu` besteht darin, dass

- `stream.cu` einen einzelnen, gutartigen STREAM-Triad-Fall misst. Bei dem alle Threads regulär und zusammenhängend auf `a`, `b` und `c` zugreifen. Der Kernel beantwortet damit die Frage: Welche Bandbreite ist bei einem sauberen Streaming-Zugriff erreichbar?
- `memory.cu` variiert dagegen gezielt das Zugriffsmuster. Derselbe einfache Rechenkern wird als *coalesced*, *strided* und *random gather* ausgeführt. Damit beantwortet der Kernel die Frage: Wie stark bricht die Bandbreite ein, wenn die Adressen innerhalb einer Warps ungünstig liegen?

1.1.2 Zugriffsmuster und effektive Bandbreite

Für den Sweep wurden 33 554 432 Elemente verwendet. Die Messwerte liegen in `patterns.csv`; die daraus erzeugte Grafik ist in Abbildung 1 dargestellt.

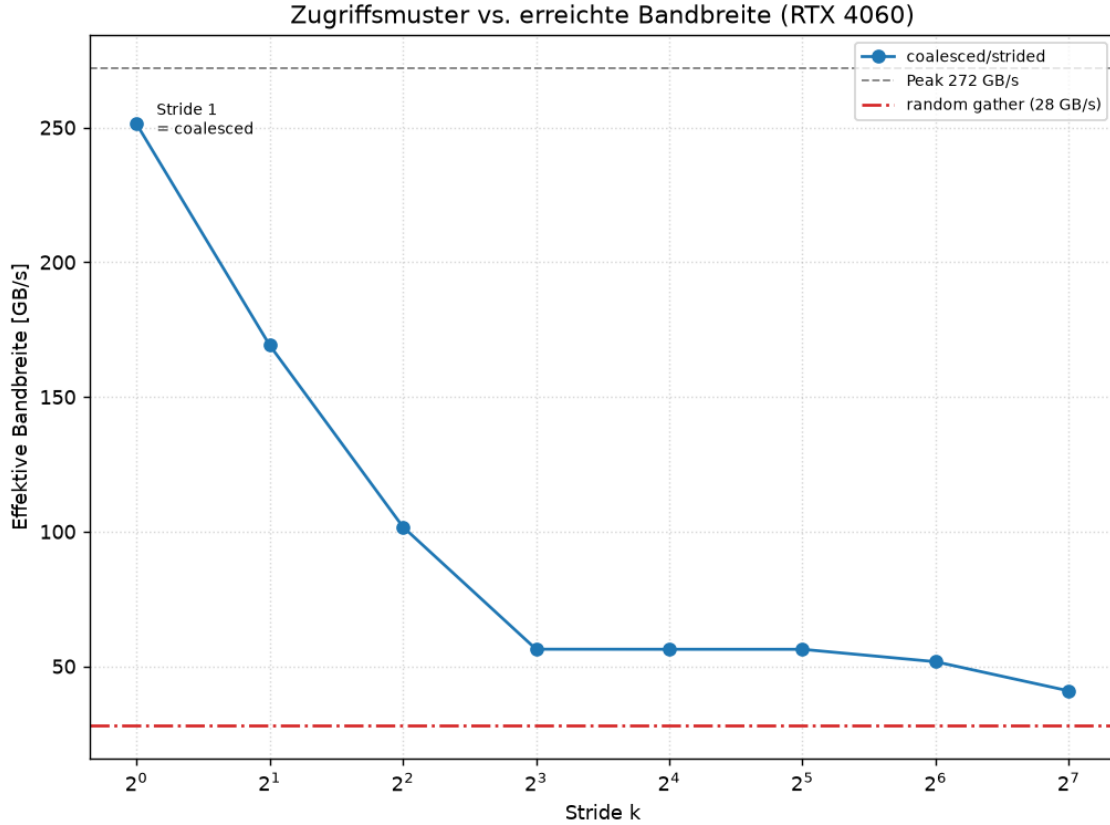


Abbildung 1: Effektive Bandbreite für coalesced, strided und Gather-Zugriffe.

Muster	Stride	GB/s	% Peak
coalesced	1	251,30	92,38
strided	2	169,15	62,18
strided	4	101,76	37,41
strided	8	56,34	20,71
strided	16	56,31	20,70
strided	32	56,30	20,70
strided	64	51,69	19,00
strided	128	40,87	15,02
random gather	-	28,03	10,30

Tabelle 1: Gemessene effektive Bandbreite für verschiedene Zugriffsmuster.

Der *coalesced* Fall ($stride=1$) erreicht mit 251,30 GB/s etwa 92 Prozent des theoretischen Peaks. Das ist der Idealfall für eine GPU: Alle 32 Lanes eines Warps lesen zusammenhängende Adressen, sodass die Hardware wenige große Speichertransaktionen bilden kann. Damit ist fast jedes übertragene Byte nutzbar.

Bei *strided* Zugriffen ($stride > 1$) sinkt die nutzbare Bandbreite dagegen deutlich. Schon bei einem *stride* von 2 fällt die nutzbare Bandbreite auf 169,15 GB/s und für einen *stride* von 4 auf 101,76 GB/s. Ab einem *stride* von 8 werden nur noch etwa 20 Prozent des Peaks (≈ 50 GB/s) erreicht. Das liegt daran, dass mit zunehmendem *stride* die Lanes eines Warps weiter auseinanderliegende Adressen lesen müssen. Dadurch müssen mehr Speichersegmente geladen werden, obwohl pro Segment nur ein kleiner Teil tatsächlich genutzt wird.

Random Gather hat mit 28,03 GB/s die geringste nutzbare Bandbreite. Jede Lane liest praktisch eine zufällige Adresse, wodurch *coalescing* und *räumliche Lokalität* fast vollständig

verloren gehen. Viele einzelne Speichertransaktionen und Cache-Misses senken dabei die effektive Bandbreite auf etwa 10 Prozent des Peaks.

2 *Occupancy* und *latency hiding*

Im zweiten Experiment wurde die Blockgröße des *coalesced* Kernels variiert. Für jede Blockgröße wurde die theoretische *occupancy* mit `cudaOccupancyMaxActiveBlocksPerMultiprocessor()` bestimmt:

$$\text{Occupancy} = \frac{\text{aktive Warps}}{\text{maximal mögliche aktive Warps}}.$$

Die Messwerte liegen in `outputs/occupancy.csv`; die Grafik ist in Abbildung 2 dargestellt.

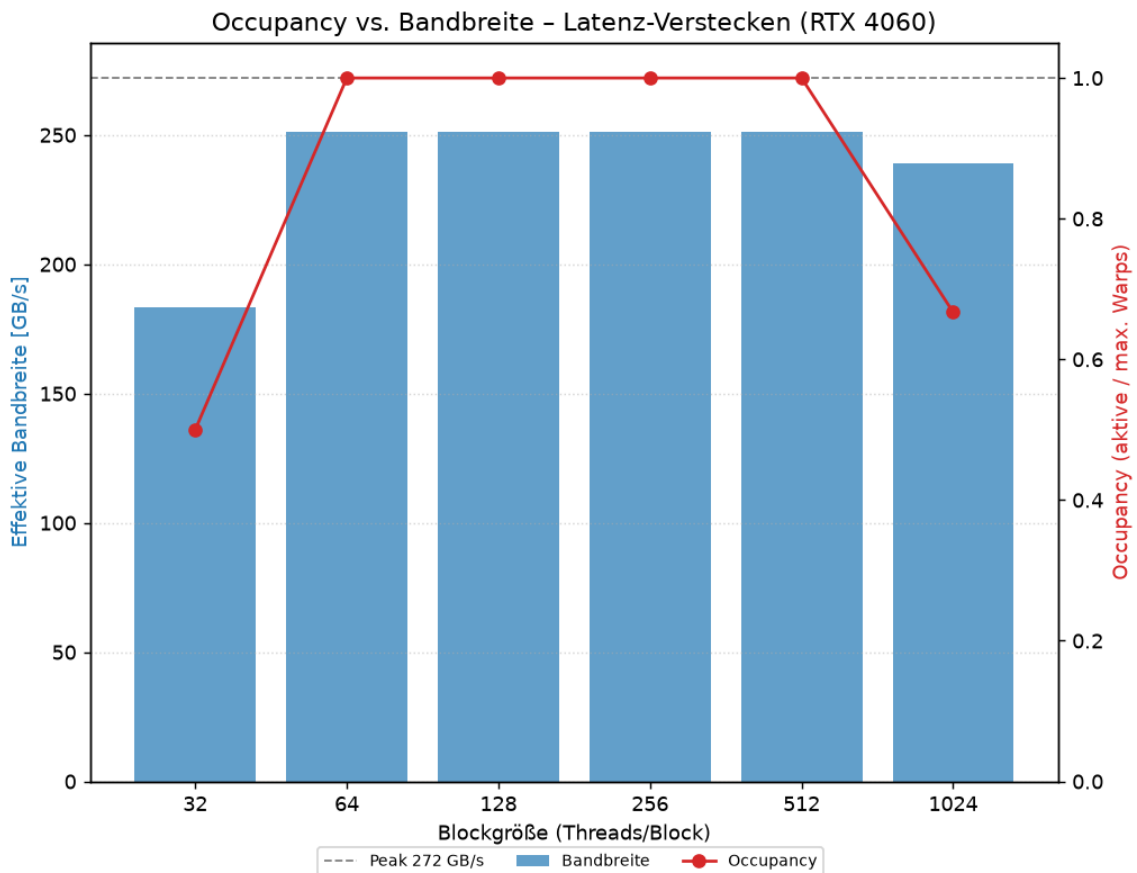


Abbildung 2: Zusammenhang zwischen Blockgröße, *Occupancy* und Bandbreite.

Blockgröße	Occupancy	GB/s	% Peak
32	0,500	183,24	67,36
64	1,000	251,30	92,38
128	1,000	251,32	92,39
256	1,000	251,33	92,39
512	1,000	251,30	92,38
1024	0,667	239,14	87,91

Tabelle 2: *Occupancy-Sweep* für den *coalesced* Kernel.

Bei einer Blockgröße von 32 liegt die *occupancy* nur bei 50% und die Bandbreite fällt auf 183,24 GB/s. Das heißt, es sind zu wenige Warps in einem *Streaming Multiprocessor (SM)*, um Speicher-Stalls vollständig zu überdecken. Ab einer Blockgröße von 64 erreicht die *occupancy* stabil fast 100%. Die Bandbreite steigt dann auf ≈ 251 GB/s und bleibt bis zu einer Blockgröße von 512 stabil. Ab einer Blockgröße von 1024 sinkt die *occupancy* wieder auf $\approx 66,7\%$, da sich insgesamt weniger große Blöcke gleichzeitig auf einem *Streaming Multiprocessor* befinden können; die Bandbreite sinkt dadurch auf 239,14 GB/s.

Das Experiment zeigt sog. *latency hiding*: Da ein einzelner Speicherzugriff mehrere Zyklen dauern kann, kann der *Warp-Scheduler* bei einem *stall* während der Verarbeitung eines Warps auf einen anderen Warp wechseln, solange sich genügend lauffähige Warps im *Streaming Multiprocessor* befinden. Dadurch wird die Speicherlatenz eines einzelnen Warps zwar nicht vermieden, aber durch Parallelität überdeckt.

2.1 GPU- und CPU-Strategien im Vergleich

Die GPU vermeidet lange Speicherlatenzen durch einen hohen Durchsatz, indem sich viele Warps gleichzeitig in den *SMs* befinden und Warp-Scheduling genutzt wird, um lange Speicherlatenzen zu maskieren. Wie Experimente zu den Zugriffsmustern *strided* und *random gather* zeigen, sind GPU Caches im Vergleich zu CPU-Caches klein und können zufällige Zugriffe nicht in gleichem Umfang abfedern. Deshalb entscheidet das Zugriffsmuster direkt über die erreichte Bandbreite.

Die CPU ist dagegen auf eine niedrige Latenz einzelner Instruktionsströme optimiert. Sie nutzt hierfür mehrstufige Cache-Hierarchien, Hardware-Prefetching, Out-of-Order-Execution und wenige starke Kerne. Sequenzielle Zugriffe können dadurch sehr schnell sein, weil Daten rechtzeitig in den Cache geladen werden. Falls zufällige Zugriffe Cache-Misses verursachen, sind diese auch für die CPU teuer.

2.2 Folgen für das Datenlayout und GPU-Programme

Aus den Messungen ergeben sich die folgenden Konsequenzen für effiziente GPU-Programme:

- Benachbarte Threads sollten benachbarte Speicheradressen lesen oder schreiben.
- Eine *Structure-of-Arrays* ist für viele GPU-Zugriffsmuster günstiger als ein *Array-of-Structures*, weil Felder dann zusammenhängend gelesen werden.
- *Random Gather* und *Scatter* sollten vermieden oder durch vorheriges Sortieren bzw. Umordnen der Daten reduziert werden.
- Daten sollten passend ausgerichtet und bei 2D-Strukturen gegebenenfalls gepaddet werden.
- Es müssen genug Threads bereitgestellt und passende Blockgrößen gewählt werden, damit sich immer genügend lauffähige Warps gleichzeitig in den *SMs* befinden können.